Com S // Cpr E // Math 4240

Introduction to High-Performance Computing

Instructor: Songting Luo

Lecture 19: Loop dependencies, threadsafe functions, & fine vs. coarse grain parallelism

Outline

1. Recall: `omp parallel` vs. `omp parallel for`
2. Loop dependencies
3. Thread-safe functions
4. Fine vs. coarse grain parallelism

Recall: `omp parallel` vs. `omp parallel for`

OpenMP parallel with do loops

Example #1:

```
1  # pragma omp parallel num_threads(thread_count)
2  for (int i=1; i<=10; i++)
3  {
4    printf(" i = %i\n",i);
5  }
```

This code will be executed on each thread. Therefore, with 2 threads, 20 lines will be printed.

Example #2:

```
1  # pragma omp parallel for num_threads(thread_count)
2  for (int i=1; i<=10; i++)
3  {
4    printf(" i = %i\n",i);
5  }
```

This code only prints 10 lines, independent of how many threads used.

Loop dependencies

Dependencies in loops

```
1  for (int i=0; i<N; i++)
2  {
3      z[i] = x[i] + y[i];
4      w[i] = cos(z[i]);
5  }
```

There is data dependence between the two statements in this loop.

The value `w[i]` cannnot be computed before `z[i]`.

However, this *can* be paralellized with a parallel for since the same thread will always execute both statements in the right order for each *i*.

Dependencies in loops

```
1  for (int i=0; i<N; i++)
2  {
3      z[i] = x[i] + y[i];
4      w[i] = cos(z[i]);
5  }
```

There is data dependence between the two statements in this loop.

The value `w[i]` cannnot be computed before `z[i]`.

However, this *can* be paralellized with a parallel for since the same thread will always execute both statements in the right order for each *i*.

```
1  # pragma omp parallel for num_threads(thread_count)
2  for (int i=0; i<N; i++)
3  {
4      z[i] = x[i] + y[i];
5      w[i] = cos(z([i]));
6  }
```

Matrix-matrix multiplication

```
1  matrix matrix_mult_v3(const matrix* A, const matrix* B)
2  {
3     const int rowsA = A->rows; const int colsA = A->cols;
4     const int rowsB = B->rows; const int colsB = B->cols;
5     assert(colsA==rowsB); matrix C = new_matrix(rowsA,colsB);
6
7     matrix Btranspose = new_matrix_v2(colsB,rowsB);
8
9  #  pragma omp parallel for
10    for (int i=1; i<=rowsA; i++)
11       for (int j=1; j<=colsB; j++)
12       { mget(Btranspose,i,j) = mgetp(B,j,i); }
13
14 #  pragma omp parallel for
15    for (int i=1; i<=rowsA; i++)
16       for (int j=1; j<=colsB; j++)
17          for (int k=1; k<=colsA; k++)
18          { mget(C,i,j) += mgetp(A,i,k)*mget(Btranspose,j,k); }
19
20    delete_matrix(&Btranspose);
21    return C;
22 }
```

This works since `mget(C,i,j)` is only modified by thread handling row `i`.

Matrix-matrix multiplication

$ISUHPC/lectures/lecture19/codes/parallel_matrix_mult/main.c

```
1  int main()
2  {
3      // Get dimensions
4      int N=0;
5      printf("\n      Input N: "); scanf("%i",&N); assert(N>0);
6
7      // Create some matrices
8      matrix A = new_matrix(N,N);
9      matrix B = new_matrix(N,N);
10
11     int num_threads = 0;
12 #   pragma omp parallel
13     num_threads = omp_get_num_threads();
14
15 #   pragma omp parallel for
16     for (int i=1; i<=N; i++)
17         for (int j=1; j<=N; j++)
18         {
19             mget(A,i,j) = (double)(i + j)/((double)(2*N));
20             mget(B,i,j) = (double)(i - j)/((double)N);
21         }
```

Matrix-matrix multiplication

$ISUHPC/lectures/lecture19/codes/parallel_matrix_mult/main.c

```
1     const double time1 = omp_get_wtime();
2     matrix C1 = matrix_mult_v2(&A,&B);
3     const double time2 = omp_get_wtime();
4     matrix C2 = matrix_mult_v3(&A,&B);
5     const double time3 = omp_get_wtime();
6
7     double err = 0.0;
8  #  pragma omp parallel for reduction (+:err)
9     for (int i=1; i<=N; i++)
10       for (int j=1; j<=N; j++)
11       {
12          double tmp = fabs(mget(C1,i,j)-mget(C2,i,j));
13 //#         pragma omp critical
14          err += tmp;
15       }
16
17    printf("\n");
```

Matrix-matrix multiplication

```
$ make
gcc -O3 -Wall -std=c99 -fopenmp -c matrix.c
gcc -O3 -Wall -std=c99 -fopenmp -c main.c
gcc -lgomp matrix.o main.o -o main.exe

$ main.exe

      Input N: 1000

  Difference (Serial)-(OpenMP) =  0.00000e+00
   Time for Serial MatMat Mult =  1.51184e+00
   Time for OpenMP MatMat Mult =  3.86960e-01
         Number of Threads Run =  8
                 Speedup Factor =  3.90698e+00
```

Loop-carried dependencies

```
1  for (int i=1; i<=N; i++)
2  {  vget(x,i) = 1.0;  }
3
4  vget(x,1) = 5.0;
5  for (int i=2; i<=N; i++)
6  {
7    vget(x,i) = vget(x,i-1);
8  }
```

There is a loop-carried data dependence in this loop.

The assignment for `i=3` must not be done before `i=2` or it may get the wrong value.

Therefore, this code cannot be parallelized.

Loop-carried dependencies

Example: Solve ODE initial value problem

$$y'(t) = 2y(t), \quad y(0) = 1$$

with Euler's method

$$y(t + \Delta t) \approx y(t) + \Delta t\, y'(t) = (1 + 2\Delta t)y(t)$$

to approximate $y(t) = e^{2t}$ for $0 \leq t \leq 5$:

```
1  const int N = 5000;        // number of time steps
2  const double Tfinal = 5.0;  // final time
3  const double dt = Tfinal/((double)N); // time-step
4
5  vget(y,1) = 1.0; // initial conditions
6
7  // time-stepping loop
8  for (int n=2; n<=N; n++)
9  {  vget(y,n) = (1.0+2.0*dt)*vget(y,n-1);  }
```

Cannot easily parallelize.

Loop-carried dependencies

```
1  vector x = new_vector(10);
2  double y = 0.0;
3
4  for (int i=1; i<=10; i++)
5  {
6    if (i==3)
7    {  y = 1.0;  }
8    vget(x,i) = y;
9  }
```

There is a loop-carried data dependence in this loop.

In serial execution: only first two elements of `x` are 0.0.

With `#pragma parallel for`:

later index (e.g., `i=6`) might be executed before `i=3`.

Thread-safe functions

Thread-safe functions

Consider this code:

```
1  vector y = new_vector(N);
2
3  # pragma omp parallel for num_threads(thread_count)
4  for (int i=1; i<=N; i++)
5  {
6    vget(y,i) = myfcn(vget(x,i));
7  }
```

Does this give the same results as the serial version?

Thread-safe functions

Consider this code:

```
1  vector y = new_vector(N);
2
3  # pragma omp parallel for num_threads(thread_count)
4  for (int i=1; i<=N; i++)
5  {
6    vget(y,i) = myfcn(vget(x,i));
7  }
```

Does this give the same results as the serial version?

Maybe not... it depends on what the function does!

If this gives the same results regards of the order threads call for different values of `i`, then the function is thread safe.

Thread-safe functions

A thread-safe function:

```
1 double function myfcn(const double x)
2 {
3   double z = exp(x);
4   return z*cos(x);
5 }
```

Executing this function for one value of `x` is completely independent of execution for other values of `x`.

Thread-safe functions

A thread-safe function:

```
1  double function myfcn(const double x)
2  {
3    double z = exp(x);
4    return z*cos(x);
5  }
```

Executing this function for one value of `x` is completely independent of execution for other values of `x`.

This function is not thread-safe:

```
1  double function myfcn(const double x)
2  {
3    double z;
4    scanf("%lf",&z);
5    return z*cos(x);
6  }
```

Will work in serial, but if threads execute in different order, will get different results.

Fine vs. coarse grain parallelism

Fine vs. coarse grain parallelism

Fine grain: parallelize at the level of individual loops, splitting work for each loop between threads.

Coarse grain: split problem up into large pieces and have each thread deal with one piece.

May need to synchronize or share information at some points.

Fine vs. coarse grain parallelism

Fine grain: parallelize at the level of individual loops, splitting work for each loop between threads.

Coarse grain: split problem up into large pieces and have each thread deal with one piece.

May need to synchronize or share information at some points.

More similar to what must be done in MPI.

Fine vs. coarse grain parallelism

Fine grain: parallelize at the level of individual loops, splitting work for each loop between threads.

Coarse grain: split problem up into large pieces and have each thread deal with one piece.

May need to synchronize or share information at some points.

More similar to what must be done in MPI.

Domain decomposition: splitting up a problem on a large domain (e.g., 3D grid) into pieces are handled separately (with suitable coupling)

Example: normalizing a vector

Compute the normalized vector

$$\frac{x}{\|x\|_1} \quad \text{with} \quad \|x\|_1 = \sum_{i=1}^{N} |x_i|$$

Fine-grain: using `parallel for` loops.

```
1  double norm = 0.0;
2
3  # pragma omp parallel for reduction(+:norm)
4  for (int i=1; i<=N; i++)
5  {  norm += fabs(vget(x,i));  }
6
7  # pragma omp parallel for
8  for (int i=1; i<=N; i++)
9  {  vget(x,i) = vget(x,i)/norm;  }
```

NOTE: must finish computing `norm` before changing any `x(i)`, so we are using an implicit barrier after the first loop.

Barrier: place in the code where we only continue once all the threads have reached this point.

Example: normalizing a vector

Another fine-grain approach, forking threads only once:

```
1      for (int i=1; i<=N; i++)
2      {  vget(x,i) = -1.0; }
3
4      double norm = 0.0;
5
6      const double time1 = omp_get_wtime();
7
8   #  pragma omp parallel num_threads(thread_count)
9      {
10  #     pragma omp for reduction(+:norm)
11        for (int i=1; i<=N; i++)
12        {  norm += fabs(vget(x,i));  }
13
14  #     pragma omp barrier  // not needed (implicit)
15
16  #     pragma omp for
17        for (int i=1; i<=N; i++)
18        {  vget(x,i) = vget(x,i)/norm;  }
19     }
```

Example: normalizing a vector

Compute the normalized vector

$$\frac{x}{\|x\|_1} \quad \text{with} \quad \|x\|_1 = \sum_{i=1}^{N} |x_i|$$

Coarse grain version:

Assign blocks of i values to each thread. Threads must:

- Compute thread's contribution to $\|x\|_1$:

$$\texttt{norm_thread} = \sum_{\texttt{istart}}^{\texttt{iend}} |x_i|$$

- Collaborate to compute the total value of $\|x\|_1$:

$$\|x\|_1 = \sum_{\text{thread}} \texttt{norm_thread}$$

- Loop over `i = istart,iend` to divide x_i by $\|x\|_i$.

Example: normalizing a vector

$ISUHPC/lectures/lecture19/codes/normalizing_vector_coarse_grain/main.c

```
1  #  pragma omp parallel num_threads(thread_count)
2     {
3        const int my_rank = omp_get_thread_num();
4
5        const int N_per_thread = N/thread_count;
6        const int istart = my_rank*N_per_thread + 1;
7        const int iend   = (my_rank+1)*(N_per_thread);
8
9        double norm_thread = 0.0;
10       for (int i=istart; i<=iend; i++)
11       {  norm_thread += fabs(vget(x,i));  }
12
13 #     pragma omp critical
14       norm += norm_thread;
15
16 #     pragma omp barrier // needed here
17
18       for (int i=istart; i<=iend; i++)
19       {  vget(x,i) = vget(x,i)/norm;  }
20    }
```

Example: shared exit criterion

System of ODEs (each equation is decoupled from the others):

$$u_m'(t) = c_m\, u_m(t) \quad \text{for} \quad t > 0, \qquad \textbf{ICs} : u_m(0) = \eta_m$$

for $m = 1, 2, \ldots, M$.

Exact solution: $u_m(t) = e^{c_m t}\eta_m$

Euler method: $u_m(t + \Delta t) \approx u_m(t) + \Delta t\, c_m\, u_m(t) = (1 + c_m\, \Delta t)\, u_m(t)$

New wrinkle: stop time stepping when any of the $u_i(t)$ values exceeds 100.

(Will certainly happen as long as $c_j > 0$ for some j.)

Example: shared exit criterion

System of ODEs (each equation is decoupled from the others):

$$u_m'(t) = c_m\, u_m(t) \quad \text{for} \quad t > 0, \qquad \textbf{ICs} : u_m(0) = \eta_m$$

for $m = 1, 2, \ldots, M$.

Exact solution: $u_m(t) = e^{c_m t}\eta_m$

Euler method: $u_m(t + \Delta t) \approx u_m(t) + \Delta t\, c_m\, u_m(t) = (1 + c_m\, \Delta t)\, u_m(t)$

New wrinkle: stop time stepping when any of the $u_i(t)$ values exceeds 100.

(Will certainly happen as long as $c_j > 0$ for some j.)

Idea: Each time step, compute `umax` = maximum value of u_m over all m and exit time-stepping if `umax` > `100`.

Each thread has a private variable `umax_thread` for the maximum value of u_m for its values of `m`.

Each thread updates shared `umax` based on its `umax_thread`. This needs to be done in critical section. Also need two barriers to make sure all threads are in synch at certain points.

```
1  #  pragma omp parallel num_threads(thread_count)
2     {
3        const int my_rank = omp_get_thread_num();
4        const int M_per_thread = M/thread_count;
5        const int mstart = my_rank*M_per_thread + 1;
6        const int mend   = (my_rank+1)*(M_per_thread);
7        for (int n=1; n<=N; n++)
8        {
9           double umax_thread = 0.0; int m_max = 0;
10 #         pragma omp single
11          umax = 0.0;
12          for (int m=mstart; m<=mend; m++)
13          {
14             vget(u,m) = (1.0+vget(c,m)*dt)*vget(u,m);
15             vget(t,m) = vget(t,m)+dt;
16             if (vget(u,m)>umax_thread)
17             {  umax_thread = vget(u,m); m_max = m;  }
18          }
19 #         pragma omp critical
20          if (umax_thread>umax)
21          {  umax = umax_thread; m_store = m_max;  }
22 #         pragma omp barrier
23          if (umax>100.0)
24          { break; }
25 #         pragma omp barrier
26       }
27    }
```

$ISUHPC/lectures/lecture19/codes/euler_shared_exit/main.c

```
1  int main(int argc, char* argv[])
2  {
3     void usage(const char* prog_name);
4     srand( time(NULL) );
5
6     if (argc != 3) { usage(argv[0]); }
7     const int thread_count = strtol(argv[1], NULL, 10);
8     const int N = strtol(argv[2], NULL, 10);
9     if (thread_count<1 || N<1 )
10    { usage(argv[0]); }
11
12    const double Tfinal = 100.0;  // final time
13    const double dt = Tfinal/((double)N); // time-step
14    const int M = 12800;
15    vector u = new_vector(M);
16    vector c = new_vector(M);
17    vector t = new_vector(M);
```

$ISUHPC/lectures/lecture19/codes/euler_shared_exit/main.c

```
1  #  pragma omp parallel for num_threads(thread_count)
2     for (int m=1; m<=M; m++)
3     {  vget(u,m) = 1.0; }
4
5  #  pragma omp parallel for num_threads(thread_count)
6     for (int m=1; m<=M; m++)
7        for (int k=1; k<=100; k++)
8        {  vget(c,m) += ((double)(rand() % 10000))/1000000.0; }
9
10    double c_max = 0.0;
11    int m_c_max = 0;
12    for (int m=1; m<=M; m++)
13    {
14       if (vget(c,m)>c_max)
15       { c_max = vget(c,m); m_c_max = m; }
16    }
17
18 #  pragma omp parallel for num_threads(thread_count)
19    for (int m=1; m<=M; m++)
20    {  vget(t,m) = 0.0; }
21
22    int m_store;
23    double umax;
24    const double time1 = omp_get_wtime();
```

```
1  #  pragma omp parallel num_threads(thread_count)
2     {
3        const int my_rank = omp_get_thread_num();
4        const int M_per_thread = M/thread_count;
5        const int mstart = my_rank*M_per_thread + 1;
6        const int mend   = (my_rank+1)*(M_per_thread);
7        for (int n=1; n<=N; n++)
8        {
9           double umax_thread = 0.0; int m_max = 0;
10 #         pragma omp single
11          umax = 0.0;
12          for (int m=mstart; m<=mend; m++)
13          {
14             vget(u,m) = (1.0+vget(c,m)*dt)*vget(u,m);
15             vget(t,m) = vget(t,m)+dt;
16             if (vget(u,m)>umax_thread)
17             {  umax_thread = vget(u,m); m_max = m;  }
18          }
19 #         pragma omp critical
20          if (umax_thread>umax)
21          {  umax = umax_thread; m_store = m_max;  }
22 #         pragma omp barrier
23          if (umax>100.0)
24          { break; }
25 #         pragma omp barrier
26       }
27    }
```